

# 프로젝트 결과보고서

## 과 제 명

|       |            |       |
|-------|------------|-------|
| 제 출 일 | 2026.06.18 |       |
| 학 과   | 정보통신공학과    |       |
| 팀 명   | 3조         |       |
| 팀 장   | 20210581   | 최 준 서 |
| 팀 원   | 20220540   | 김 가 온 |
|       | 20220490   | 오 성 준 |
|       | 20210558   | 홍 민 혁 |
| 지도교수  | 은성배 교수님    |       |

# 목차

1. 개요
  - 1.1 시스템 기본 구조
2. 관련 기술 현황
  - 2.1 관련 기술, 시스템, 장치 등의 상세한 조사 및 분석
  - 2.2 기존 상용 시스템 소개 (Epaper)
  - 2.3 관련 특허 소개
  - 2.4 관련 논문 자료 조사
  - 2.5 관련 기술, 시스템의 분석표
3. 문제 정의
4. 아이디어 창출 및 평가
  - 4.1 아이디어 창출
  - 4.2 아이디어 평가
5. 설계
  - 5.1 시스템 구성도
  - 5.2 사용자 시나리오
  - 5.3 각 장치의 설계도 및 설명
  - 5.4 기대 효과
6. 구현
  - 6.1 활용 부품 내역
  - 6.2 기존 설계 내용과 다르게 C보드를 추가한 이유
  - 6.3 최종 구현 시스템 사진 등을 포함한 시스템 설명
  - 6.4 동작 모습의 동영상
7. 성능평가
  - 7.1 추정 성능 실험
  - 7.2 추정 실험 데이터 및 분석
  - 7.3 정량 및 정성 평가
8. 현실적 제한조건 반영
9. 팀 역할 소개
- 10.참고문헌
- 11.부록

# 1. 개요

## 1.1 저전력 E-Paper 디스플레이를 활용한 스마트 홈 캘린더 + 탁상용 선풍기(추가된 내용)

- 시스템의 기본 구조는 다음과 같다.

| A 보드           | B 보드 (허브)           | C 보드              |
|----------------|---------------------|-------------------|
| e-Paper<br>저전력 | WiFi-API-센서<br>웹 설정 | LED·선풍기<br>서보·초음파 |

\* C 보드 추가 이유 후에 설명

- A 보드 (캘린더 디스플레이) <-> B 보드 (데이터 수집 허브) :

책상 위에서 날짜·날씨·대기질·AI 생활 조언을 보여주는 스마트 디스플레이다.  
역할이 다른 두 ESP32 보드를 ESP-NOW로 연결해,  
한쪽은 인터넷 데이터를 모으는 게이트웨이로,  
다른 쪽은 배터리로 오래 버티는 저전력 표시 단말로 분리한 것이 핵심 설계이다.

- C 보드 (탁상용 선풍기) <-> B 보드 (데이터 수집 허브) :

거리, 환경에 반응하는 LED 조명과 선풍기를 제어하는 역할로,  
디폴트 모드 / 자동 모드로 나누어져 있다.

다음 페이지에서 계속

## 2. 관련 기술 현황

### 2.1 관련 기술, 시스템, 장치 등의 상세한 조사 및 분석

#### - [ESP32] ESP-NOW 통신 기술 설명

ESP-NOW는 Espressif가 정의한 커넥션리스(connectionless) 무선 통신 프로토콜로, OSI 5개 상위 계층을 데이터링크 계층 하나로 축약해 라우터 없이 기기 간 직접 통신을 수행한다.

공식 자료 기준 핵심 특성은 송신 전력 최대 +21dBm으로 개활지 약 200m 통신, 응답 지연 약 150ms 이내, WiFi/BLE와 공존 가능, ECDH-AES 기반 암호화 지원, 그리고 윈도우 동기화(window synchronization)를 통한 저전력 동작이다.

#### - [Crowpanel epaper 4.2] Epaper 기술 설명

전자종이(E-paper, Electronic Paper)는 실제 종이에 잉크로 인쇄한 것과 비슷한 모습을 전자적으로 구현한 디스플레이이다.

가장 널리 쓰이는 방식이 미국 E Ink Corporation이 상용화한 전기영동(電氣泳動, Electrophoretic Display, EPD) 방식이며, 흔히 "E-ink"라는 상표명으로도 불린다.

LCD나 OLED가 빛을 스스로 내거나 백라이트로 비추는 발광형 디스플레이인 것과 달리, 전자종이는 빛을 내지 않고 주변의 빛을 반사해서 보이는 반사형 디스플레이이다. 그래서 종이를 보는 것과 같은 느낌을 주고, 햇빛 아래에서도 잘 보인다.

#### - [Crowpanel epaper 4.2] Epaper 동작 원리

전기영동 방식의 화면은 수많은 미세한 캡슐(마이크로캡슐)로 채워져 있다. 각 캡슐 안에는 투명한 액체와 함께 음(-)전하를 띤 흰색 입자와 양(+)전하를 띤 검은색 입자가 떠 있다. 전극으로 전기장을 걸어 주면 전하를 띤 입자들이 위(표면)로 이동하는데, 흰 입자를 위로 올리면 그 픽셀은 흰색으로, 검은 입자를 올리면 검은색으로 보인다. 비유하자면 작은 어항 안에서 흰 구슬과 검은 구슬을 자석으로 위아래로 끌어올려 색을 바꾸는 것과 비슷하다.

핵심은 한 번 입자를 이동시켜 글자나 그림을 만들고 나면, 전원을 꺼도 입자가 그 자리에 그대로 머문다는 점이다. 이를 쌍안정성(bistability)이라고 한다. 즉 화면을 "바꿀 때"만 전기가 필요하고, "보여주고 있는 동안"에는 전기가 거의 들지 않는다.

#### - [Google Gemini / Open Meteo] 외부 API

Open-Meteo는 전 세계 기상 예보를 제공하는 오픈소스 무료 날씨 API다.

가장 큰 특징은 비상업적 용도에 한해 API 키가 필요 없다는 점이어서, 본 시스템처럼 가입·인증 절차 없이 곧바로 호출할 수 있다.

Gemini는 구글이 개발한 대규모 언어 모델(LLM)이며, Gemini API는 이 모델을 HTTP 요청으로 호출해 텍스트를 생성받는 클라우드 서비스이다.

## 2.2 기존 상용 시스템 소개 (Epaper)

### 1. 전자책 단말기

전자종이의 가장 대중적인 응용 분야이다. 종이책 같은 가독성과 며칠~몇 주씩 가는 배터리가 핵심 장점이다.

### 2. 전자가격 표시기

대형마트·편의점 매대에서 종이 가격표 대신 쓰는 소형 전자종이 라벨이다. 가격을 무선으로 일괄 변경할 수 있고, 한 번 표시하면 전력이 거의 안 들어 코인 배터리로 수년간 동작한다.

## 2.3 관련 특허 소개

### - 전자종이의 특허 (US5961804A — Microencapsulated electrophoretic display)

발명자는 Joseph Jacobson, Barrett Comiskey, Jonathan Albert 세 사람이고, 출원인(양수인)은 MIT(매사추세츠 공과대학)이다.

1997년 3월 18일 출원되어 1999년 10월 5일 등록되었고,

법적 상태는 "Expired - Lifetime"으로 이미 만료(2017년경)되었다.

즉 이 원천 특허 자체는 현재 권리가 소멸해 누구나 그 기본 기술을 사용할 수 있다.

### - 스마트홈 센서 자동 제어 특허 (US20160259308A1 — Smart-home automation system (Google/Nest 계열))

센서로 관측한 데이터에 기반해 가정의 동작 정책을 제안하거나 자동 실행하는 시스템을 다룬다. 본 시스템의 AUTO 모드(B의 센서값으로 C가 출력을 자동 결정)와 개념적으로 맞닿아 있다.

### - 스마트홈 센서 자동 제어 특허 (US7152040B1 — Electronic shelf label (전자 선반 라벨))

소비자에 부착되는 전자 가격표로서, 선택적으로 전자 디스플레이를 포함하는 무선 데이터 태그를 정의한다.

무선으로 가격 정보를 받아 표시하고 저전력으로 장기 동작하는 구조로, "서버가 무선으로 다수의 저전력 표시 단말에 정보를 보내 화면을 갱신한다"는 점에서 본 시스템의 'B 게이트웨이 → A e-paper 단말' 데이터 흐름과 개념적으로 매우 유사하다.

### - 전체 정리

|                                                                                       |                                                                                                                         |
|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| 전자종이의 특허<br>(US5961804A<br>— Microencapsulated electrophoretic display)               | <a href="https://patents.google.com/patent/US5961804A/en">https://patents.google.com/patent/US5961804A/en</a>           |
| 스마트홈 센서 자동 제어 특허<br>(US20160259308A1 — Smart-home automation system (Google/Nest 계열)) | <a href="https://patents.google.com/patent/US20160259308A1/en">https://patents.google.com/patent/US20160259308A1/en</a> |
| 스마트홈 센서 자동 제어 특허<br>(US7152040B1 — Electronic shelf label (전자 선반 라벨))                 | <a href="https://patents.google.com/patent/US7152040B1/en">https://patents.google.com/patent/US7152040B1/en</a>         |

## 2.4 관련 논문 자료 조사

### - [ESP-NOW] 통신 프로토콜 성능 비교 연구

(Comparative Performance Study of ESP-NOW, Wi-Fi, Bluetooth Protocols based on Range, Transmission Speed, Latency, Energy Usage and Barrier Resistance)

통신 프로토콜 비교 분야에서는 IEEE에 게재된 "Comparative Performance Study of ESP-NOW, Wi-Fi, Bluetooth Protocols based on Range, Transmission Speed, Latency, Energy Usage and Barrier Resistance"(IEEE Xplore Doc. 9573246)가 대표적이다.

이 논문은 ESP-NOW가 저부하·실시간 응답 시나리오에서 WiFi/BLE 대비 낮은 지연과 낮은 에너지를 보임을 실측으로 제시한다. 또한 "Indoor Performance Evaluation of ESP-NOW", "Performance Evaluation of ESP-NOW for Low-Latency Indoor IoT Systems"는 실내 환경에서 거리·장애물에 따른 지연·패킷손실 특성을 다루며, 본 코드의 채널 스캔·재시도 설계의 정당성을 뒷받침한다. Espressif 개발자 블로그의 "Data transmission reliability over ESP-NOW protocol in indoor environments"도 신뢰성 설계 참고자료로 유용하다.

논문 주소 : <https://ieeexplore.ieee.org/document/9573246/>

## 2.5 관련 기술, 시스템의 분석표

| 항목                  | 특허 A<br>(전자종이)           | 특허 B<br>(전자가격표시기/ESL)    | 논문 A (ESP-NOW<br>비교연구) | 특허 C<br>(스마트홈센서<br>자동제어) |
|---------------------|--------------------------|--------------------------|------------------------|--------------------------|
| 경제성                 | ○<br>(원천특허 만료, 무료<br>활용) | △<br>(무선 라벨 인프라 비용)      | ×<br>(비해당)             | △<br>(센서·허브 구축 비용)       |
| 목표 효과<br>(정보 표시·제어) | △<br>(표시 원리만 제공)         | ○<br>(무선 정보 표시 단말<br>정의) | △<br>(통신 근거 제공)        | ○<br>(센서 기반 자동 제어<br>정의) |
| 저전력성                | ○<br>(쌍안정성 정의)           | ○<br>(저전력 장기 동작 라벨)      | ○<br>(저전력 통신 입증)       | ×<br>(비해당)               |
| 통신 신뢰성              | ×<br>(비해당)               | ○<br>(서버→다수 단말 무선<br>갱신) | ○<br>(실내 지연·손실 분석)     | ○<br>(유무선 통신)            |
| 확장성                 | △<br>(컬러·인쇄 확장)          | ○<br>(대규모 다중 라벨 관리)      | ○<br>(다대다 토폴로지)        | ○<br>(다수 기기 정책 적용)       |
| 인터넷/<br>클라우드 연동     | ×<br>(비해당)               | △<br>(서버 연동 갱신)          | ×<br>(비해당)             | ○<br>(클라우드 정책<br>제안·실행)  |
| 표시 가독성              | ○<br>(반사형 비발광 정의)        | ○<br>(전자종이 가격표 표시)       | ×<br>(비해당)             | ×<br>(비해당)               |
| 자동화/지능화             | ×<br>(비해당)               | ×<br>(표시 위주, 자동판단<br>없음) | ×<br>(비해당)             | ○<br>(자동화 가능)            |
| 안정성<br>(통신두절 대응)    | ×<br>(비해당)               | △<br>(단말 표시 유지)          | △<br>(간섭 영향 분석)        | ○<br>(안정장치 마련)           |

### 3. 문제 정의

이 프로젝트는 책상 위에서 매일 사용하는 종이 캘린더의 한계에서 출발하였다. 기존 종이 캘린더는 날짜와 일정만을 정적으로 표시할 뿐, 그날의 날씨·미세먼지·온습도 같은 실시간 환경 정보를 전혀 담지 못한다. 또한 사용자가 직접 매달 종이를 넘기거나 글씨를 써넣어야 하며, 한 번 인쇄된 내용은 갱신이 불가능하다.

그렇다고 이 정보들을 스마트폰이나 일반 LCD 디스플레이로 대체하려 하면 새로운 문제가 발생한다. LCD나 태블릿은 정보를 항상 띄워 두려면 전력 소모가 커서 배터리로 오래 운용할 수 없고, 화면이 발광형이라 눈이 쉽게 피로하며 종이 같은 자연스러운 가독성을 주지 못한다. 또한 매번 사람이 직접 앱을 켜서 확인해야 하므로 종이 캘린더가 주던 "고개만 돌리면 항상 보이는" 즉시성도 사라진다.

따라서 이 프로젝트의 구체적 문제는 다음과 같이 정의된다. "종이 캘린더처럼 전력 없이도 항상 정보가 떠 있고 눈이 편안하면서도, 종이 캘린더가 하지 못하는 실시간 날씨·환경 정보 갱신과 자동화를, 저전력 제약 안에서 어떻게 동시에 구현할 것인가"이다. 즉 A 보드(표시 단말)는 "종이의 장점은 살리고 한계는 극복하는 디스플레이"를 목표로 하고, B 보드(게이트웨이)는 "저전력 단말을 대신해 인터넷·클라우드와 환경 데이터를 책임지는 두뇌"를 목표로 하여 탄생하였다.

### 4. 아이디어 창출 및 평가

#### 4.1. 아이디어 창출

- 문제를 해결하기 위한 구체적인 아이디어 창출 과정 제시 SCAMPER

|                                          |                                                                                                                                                                   |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>S</b><br>(Substitute, 대체)             | 종이 매체를 전자종이(e-paper)로 대체하는 단계이다. 종이의 가독성과 "전원 없이 표시 유지"라는 장점은 그대로 살리되, 인쇄된 고정 내용을 무선으로 갱신 가능한 화면으로 바꾸었다. 이것이 A 보드에 전자종이를 채택한 직접적 근거이다.                            |
| <b>C</b><br>(Combine, 결합)                | 단순 날씨 표시 기능에 날씨·미세먼지·온습도·AI 생활 조언을 결합하는 단계이다. 종이 캘린더가 담지 못하던 환경 정보를 한 화면에 통합하였고, 이 정보들을 수집하기 위해 인터넷·센서를 담당하는 B 보드가 필요해졌다.                                         |
| <b>A</b><br>(Adapt, 응용)                  | 전자가격표시기(ESL)의 "서버가 무선으로 다수의 저전력 표시 단말을 갱신하는" 구조를 가정용으로 응용한 단계이다. B 보드가 서버 역할을, A 보드가 저전력 표시 단말 역할을 맡는 게이트웨이-단말 분리 구조가 여기서 도출되었다.                                  |
| <b>M</b><br>(Modify, 수정)                 | 표시 단말의 전력 운용 방식을 근본적으로 바꾼 단계이다. 항상 켜 두는 대신 평소에는 deep sleep으로 잠들었다가 버튼이나 정해진 시각에만 깨어나 갱신하도록 하여, 종이 캘린더에 준하는 장기 무충전 운용을 가능하게 하였다.                                   |
| <b>P</b><br>(Put to another use, 다른 용도)  | 캘린더라는 단일 용도를 넘어 날씨 대시보드·대기질 모니터·AI 비서 단말로 화면 모드를 확장한 단계이다. 같은 하드웨어가 버튼 조작으로 여러 정보 화면을 제공한다.                                                                       |
| <b>E</b><br>(Eliminate, 제거)              | 저전력 단말에서 전력을 많이 쓰는 요소를 제거한 단계이다. A 보드에서 상시 WiFi 연결과 직접적인 클라우드 호출을 제거하고, 그 역할을 모두 B 보드로 이관하였다. 또한 라우터 의존을 제거하기 위해 ESP-NOW 직접 통신을 채택하였다.                            |
| <b>R</b><br>(Reverse/Rearrange, 역발상·재배치) | "사람이 정보를 찾아가는" 방식을 "정보가 사람에게 미리 준비되는" 방식으로 뒤집은 단계이다. 정해진 기상 시각(wakeHour)에 단말이 스스로 깨어나 그날 정보를 미리 갱신해 두며, 날씨 수치를 사람이 해석하던 부담을 AI가 대신 한 줄 조언으로 바꾸어 주는 역할 재배치가 이루어졌다. |

- 도출된 핵심 아이디어 정리

| A 보드           | B 보드 (허브)           | C 보드 (후에 설명)      |
|----------------|---------------------|-------------------|
| e-Paper<br>저전력 | WiFi-API·센서<br>웹 설정 | LED·선풍기<br>서보·초음파 |

| 출발점(종이 캘린더의 한계) | SCAMPER 단계  | 도출된 아이디어                  | 담당 보드 |
|-----------------|-------------|---------------------------|-------|
| 갱신 불가능한 인쇄 내용   | 대체(S)       | 무선 갱신형 전자종이 화면            | A     |
| 날짜만 표시, 환경정보 없음 | 결합(C)       | 날씨·미세먼지·AI 조언 통합 표시       | A·B   |
| 사람이 직접 관리       | 응용(A)       | ESL식 게이트웨이-단말 분리          | B·A   |
| 전력·관리 부담        | 수정(M)·제거(E) | deep sleep + 인터넷 역할 B로 이관 | A·B   |
| 단일 용도           | 다른 용도(P)    | 다중 화면 모드 확장               | A     |
| 수치 해석의 어려움      | 역발상(R)      | AI가 수치를 생활 조언으로 변환        | B     |

## 4.2. 아이디어 평가

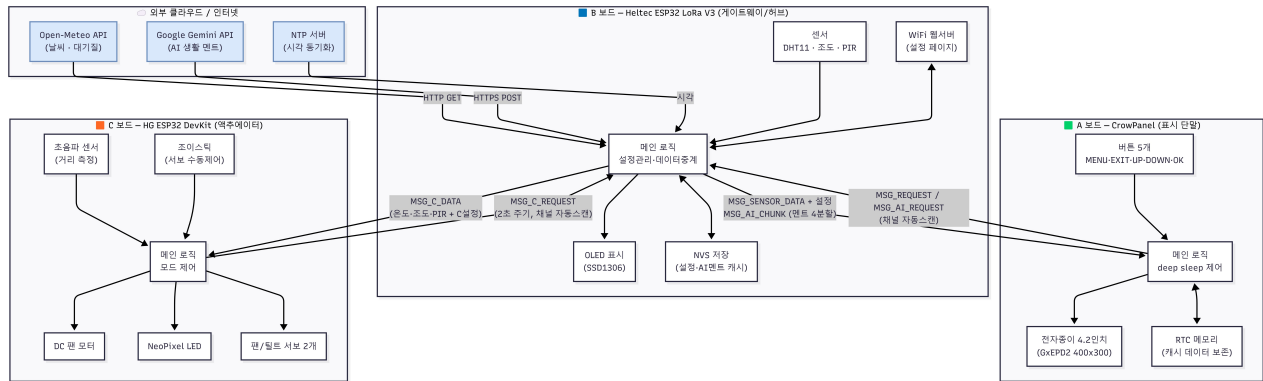
- 아이디어 평가표

| 평가 항목            | 가중치  | 종이<br>캘린더 | 일반<br>LCD 방식 | Epaper<br>(A+B) | 평가 근거                                |
|------------------|------|-----------|--------------|-----------------|--------------------------------------|
| 정보 즉시성(항상<br>표시) | 20%  | 5         | 3            | 5               | 전자종이 쌍안정성으로 전원 없이 상시 표시              |
| 실시간 정보 갱신        | 20%  | 1         | 5            | 3               | 본 시스템은 딥슬립하므로<br>완전한 실시간 정보 갱신은 불가능  |
| 저전력·장기 운용        | 20%  | 5         | 1            | 4               | A는 deep sleep으로 장기 운용,<br>B는 상시전원 필요 |
| 가독성·눈 편안함        | 10%  | 5         | 3            | 5               | 반사형 전자종이로 종이 수준 가독성                  |
| 정보 해석·실용성        | 15%  | 1         | 5            | 5               | AI가 날씨 수치를 생활 조언으로 변환                |
| 통신·동작 신뢰성        | 10%  | 5         | 4            | 4               | 채널 스캔·캐시 폴백 적용,<br>무선 한계 일부 존재       |
| 경제성              | 5%   | 5         | 3            | 3               | 게이트웨이·부품 비용 발생                       |
| 가중 합계(5점 만점)     | 100% | 3.60      | 3.10         | 3.90            | Epaper 종합 최우수                        |



# 5. 설계

## 5.1 시스템 구성도



잘 안 보일 경우 참고

<https://mermaid.ai/d/937ddfdad-3799-4be6-a40c-9a3f137811af>

이 구성도의 핵심은 B 보드를 중심으로 한 게이트웨이 구조이다. 인터넷에 연결되는 보드는 B 하나뿐이며, B가 Open-Meteo(날씨·대기질), Gemini(AI 멘트), NTP(시각)를 모두 HTTP/HTTPS로 호출해 데이터를 수집·가공한 뒤 NVS에 캐시한다.

## 5.2 사용자 시나리오

| A 보드           | B 보드 (허브)           | C 보드 (추가 내용)      |
|----------------|---------------------|-------------------|
| e-Paper<br>저전력 | WiFi-API·센서<br>웹 설정 | LED·선풍기<br>서보·초음파 |

- 시나리오 1: 아침 자동 기상 (A 보드 <-> b 보드)

|   |                                                                              |
|---|------------------------------------------------------------------------------|
| 1 | 사용자가 설정한 기상 시각(기본 7시, wakeHour)이 되면 A 보드가 스스로 deep sleep에서 깨어난다.             |
| 2 | A는 별도 조작 없이 B 보드에게 그날의 날씨·날씨·미세먼지·AI 멘트를 요청하고, 받은 정보로 전자종이 화면을 갱신한 뒤 다시 잠든다. |
| 2 | A는 별도 조작 없이 B 보드에게 그날의 날씨·날씨·미세먼지·AI 멘트를 요청하고, 받은 정보로 전자종이 화면을 갱신한 뒤 다시 잠든다. |

\*즉 사용자가 정보를 찾으러 가는 것이 아니라, 정보가 미리 준비되어 있는 것이 핵심 경험이다.

- 시나리오 2: 평상시 화면 확인 (수동 조작) (A 보드 <-> b 보드)

|   |                                                                                                          |
|---|----------------------------------------------------------------------------------------------------------|
| 1 | 낮 동안 사용자가 날씨가 궁금해 A 보드의 버튼을 누른다. deep sleep 중이던 A가 즉시 깨어나 B에게 최신 데이터를 요청하고 화면을 갱신한다.                     |
| 2 | 사용자는 UP/DOWN 버튼으로 캘린더 → 시간별 날씨 → 대기질+AI 멘트 화면을 전환하고, 캘린더 화면에서는 MENU 버튼으로 일(day) 보기와 월(month) 달력 보기를 오간다. |

|   |                                                                                |
|---|--------------------------------------------------------------------------------|
| 3 | EXIT 버튼을 누르면 시스템 정보(System Info) 화면으로 들어가 배터리·통신 채널·마지막 동기화 시각 등을 확인할 수 있다.    |
| 4 | 일정 시간(기본 10초, sleepTimeoutSec) 동안 조작이 없으면 A는 자동으로 캘린더 화면으로 돌아가 다시 잠들어 전력을 아낀다. |

- 시나리오 3: 게이트웨이가 꺼져 있을 때 (통신 실패 폴백) (A 보드 <-> b 보드)

|   |                                                                                              |
|---|----------------------------------------------------------------------------------------------|
| 1 | 사용자가 외출 중이라 B 보드(게이트웨이)의 전원이 꺼져 있는 상황을 가정한다.<br>A 보드가 버튼으로 깨어나 B에게 요청을 보내지만 응답이 없다.          |
| 2 | 이때 A는 마지막 성공 채널부터 1~13 채널까지 자동으로 스캔하며 B를 찾고, 그래도 응답이 없으면 RTC 메모리에 보관된 마지막 데이터를 그대로 화면에 표시한다. |
| 3 | 화면 우측 상단에는 "CACHED" 배지와 마지막 동기화 시각이 함께 표시되어, 사용자는 지금 보는 정보가 실시간이 아니라 캐시된 것임을 한눈에 알 수 있다.     |

- 시나리오 4: 설정 변경 (B 보드 웹 페이지) (A 보드 <-> b 보드)

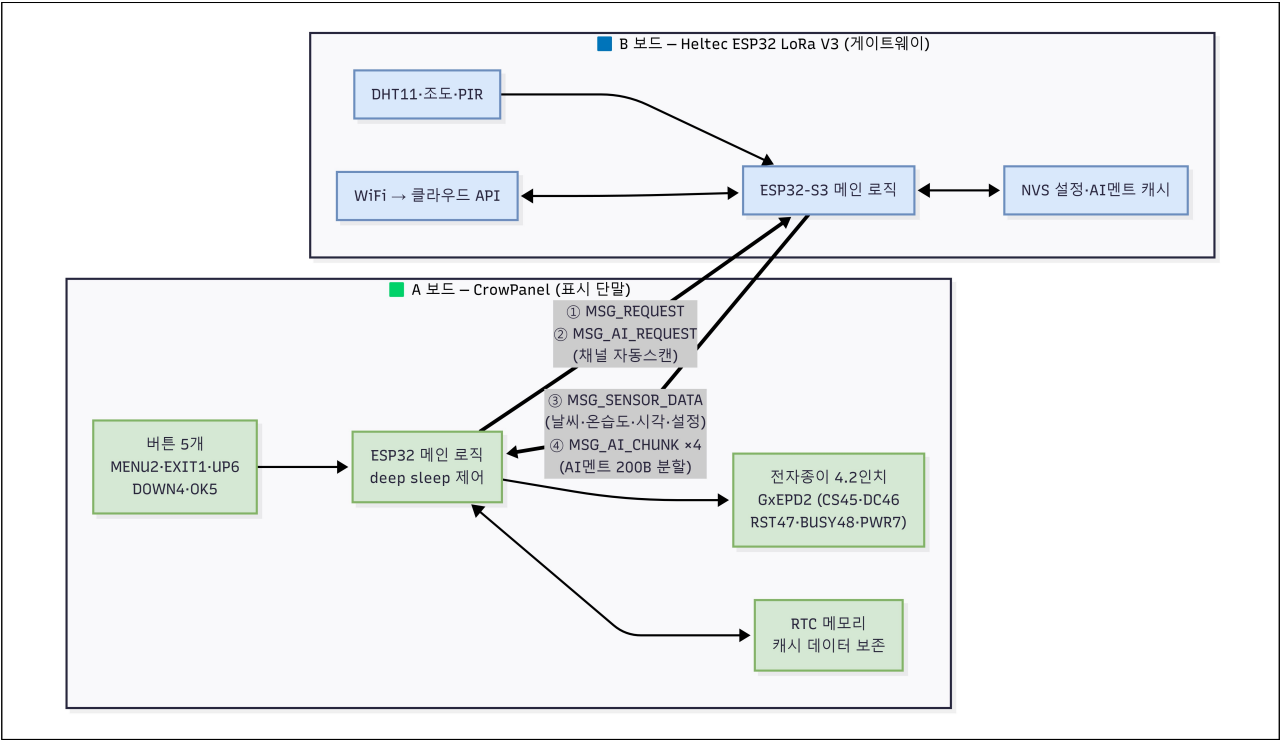
|   |                                                                                                                                                                |
|---|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | 사용자가 날씨 위치를 바꾸고 싶거나, AI 멘트 기능을 끄고 싶다. 스마트폰으로 B 보드가 제공하는 WiFi 웹 설정 페이지에 접속해 위도·경도, 기상 시각, 화면 대기시간, AI 멘트 on/off 등을 변경하고 저장한다.                                   |
| 2 | B는 이 설정을 NVS에 저장하면서 버전(configVersion)을 올린다. 다음에 A 보드가 깨어나 데이터를 요청할 때 B가 새 설정을 함께 보내고, A는 버전이 높을 때만 설정을 갱신해 적용한다. 즉 사용자는 단말을 직접 만지지 않고도 한곳에서 모든 기기 설정을 바꿀 수 있다. |

- 시나리오 5: 환경 자동 제어 (선풍기) (A 보드 <-> C 보드)

|   |                                                                                                                                                                                          |
|---|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | C 보드는 두 가지 모드로 책상 주변의 팬과 LED를 제어한다.                                                                                                                                                      |
| 2 | 기본 모드에서는 초음파 센서로 사람의 접근을 감지하여, 사용자가 책상에 앉으면 설정된 출력(LED+팬, 팬만, LED만 중 선택)에 따라 조명·팬이 켜지고 자리를 비우면 자동으로 꺼진다.<br>또한 조이스틱으로 팬·조명의 방향을 직접 조절할 수 있고, 조이스틱 버튼을 길게 누르면 가장 가까운 대상을 향해 자동으로 방향을 맞춘다. |
| 3 | AUTO 모드로 바꾸면(B 보드 웹 페이지에서 설정) 거리 대신 B 보드가 측정한 온도·조도·PIR 값을 기준으로 동작한다. 사람이 감지되고 실내가 더우면(28도 이상) 팬이, 어두우면 LED가 켜진다. 단, B와의 통신이 10초 이상 끊기면 오작동을 막기 위해 모든 출력을 안전하게 끈다(페일세이프).                |

5.3 각 장치의 설계도 및 설명

- A 보드 <-> B 보드

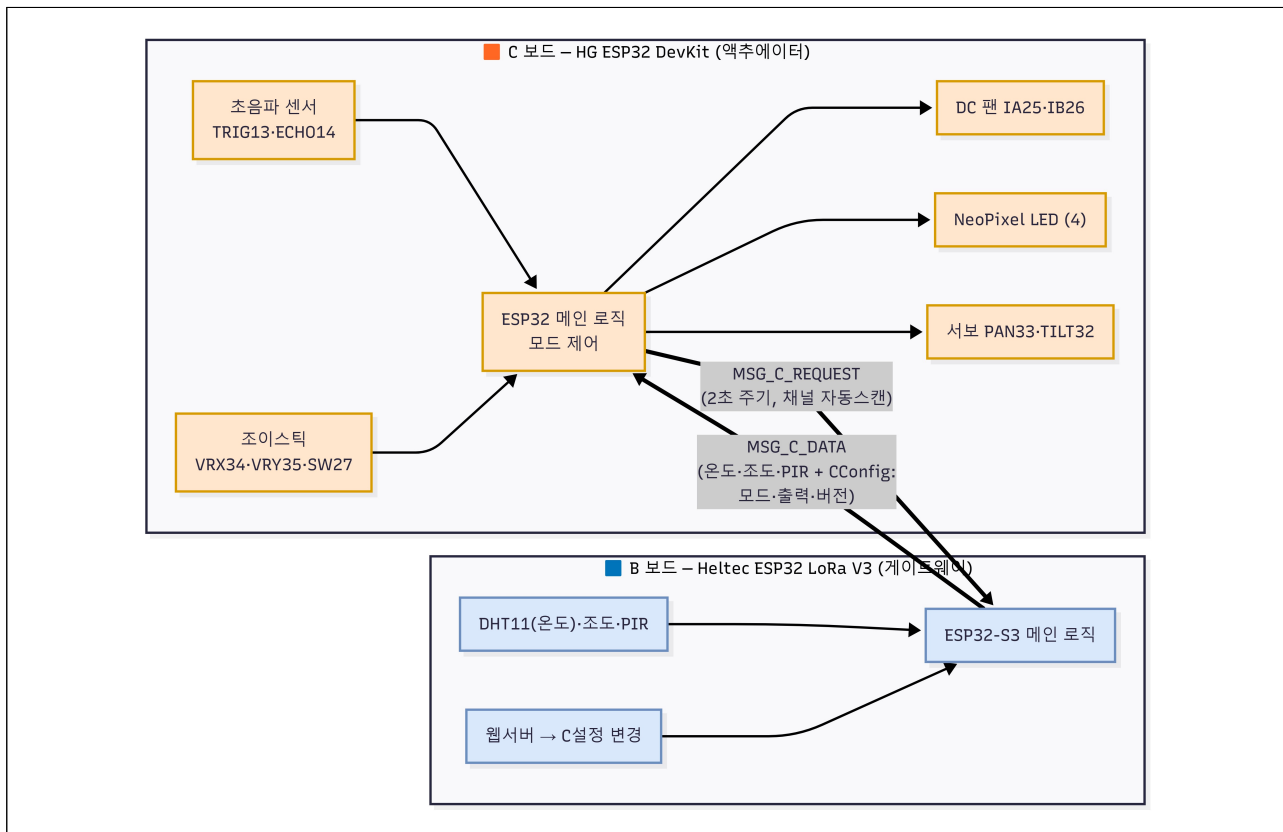


A 보드는 평소 deep sleep 상태이며, 버튼(EXT0/EXT1 wakeup) 또는 타이머로 깨어난다. 깨어나면 채널 자동 스캔으로 B를 찾아 MSG\_REQUEST(센서·날씨 요청)와 MSG\_AI\_REQUEST(AI 멘트 요청)를 보낸다.

B는 Message 구조체에 온습도·시각·시간별 날씨·대기질·현재 설정(BoardConfig)을 담아 MSG\_SENSOR\_DATA로 응답하고, AI 멘트는 ESP-NOW 250바이트 제약 때문에 200바이트씩 4개의 MSG\_AI\_CHUNK로 나누어 전송한다. A는 받은 데이터를 전자종이에 표시하고 RTC 메모리에 캐시한 뒤 다시 잠든다. B의 설정 버전(configVersion)이 높을 때만 A가 설정을 갱신한다.

| 기능 분류 | 신호명       | GPIO   | 연결 부품          | 비고                |
|-------|-----------|--------|----------------|-------------------|
| 센서    | PIN_DHT   | GPIO 1 | DHT11 데이터      | 온습도 센서            |
| 센서    | PIN_LIGHT | GPIO 2 | 조도 센서 (analog) | analogRead, 12bit |
| 센서    | PIN_PIR   | GPIO 3 | PIR 모션 센서      | 디지털 입력            |

## - C 보드 <-> B 보드



C 보드는 2초 주기로 채널 자동 스캔을 통해 B에게 MSG\_C\_REQUEST를 보낸다.

B는 자신의 온도·조도·PIR 값과 C의 동작 설정(CConfig: 모드·출력·버전)을 담아 MSG\_C\_DATA로 응답한다.

C는 받은 설정에 따라 두 모드로 동작한다.

기본(DEFAULT) 모드에서는 초음파 거리로 사람 접근을 판단해 팬·LED를 제어하고,  
 AUTO 모드에서는 B의 온도·조도·PIR 값을 기준으로 동작한다.

조이스틱 서보 제어는 모드와 무관하게 항상 작동하며, B와 10초 이상 통신이 끊기면 모든 출력을 끄는  
 페일세이프가 동작한다. C의 동작 설정은 B 웹 페이지에서 변경되어 버전과 함께 전파된다.

| 기능 분류 | 신호명            | GPIO    | 연결 부품          | 비고               |
|-------|----------------|---------|----------------|------------------|
| LED   | LED_PIN        | GPIO 4  | NeoPixel (10개) | WS2812 데이터       |
| 초음파   | TRIG_PIN       | GPIO 13 | 초음파 Trig       | 출력               |
| 초음파   | ECHO_PIN       | GPIO 14 | 초음파 Echo       | 입력               |
| 팬 모터  | MOTOR_IA       | GPIO 25 | 모터 드라이버 IA     | analogWrite(PWM) |
| 팬 모터  | MOTOR_IB       | GPIO 26 | 모터 드라이버 IB     | 방향 제어            |
| 조이스틱  | JOY_SW         | GPIO 27 | 조이스틱 버튼        | INPUT_PULLUP     |
| 조이스틱  | JOY_VRX        | GPIO 34 | 조이스틱 X축        | analog, 입력전용     |
| 조이스틱  | JOY_VRY        | GPIO 35 | 조이스틱 Y축        | analog, 입력전용     |
| 서보    | SERVO_TILT_PIN | GPIO 32 | 틸트 서보          | PWM, 500~2400us  |
| 서보    | SERVO_PAN_PIN  | GPIO 33 | 팬(좌우) 서보       | PWM, 500~2400us  |

## 5.4 기대 효과

본 시스템은 종이 캘린더의 장점인 상시 표시와 저전력을 유지하면서, 종이가 제공하지 못하던 실시간 날씨·대기질 정보와 AI 생활 조언을 더해 사용자의 일상 정보 확인 편의를 높인다.

전자종이와 deep sleep 설계로 1회 충전만으로 장기간(약 1년 내외) 운용이 가능해, 잦은 충전·관리 부담 없이 친환경적으로 사용할 수 있다.

다음 페이지에서 계속

## 6. 구현

### 6.1 활용 부품 내역

| A 보드                    |                   | B 보드             |           | C 보드                  |                  |
|-------------------------|-------------------|------------------|-----------|-----------------------|------------------|
| CrowPanel<br>epaper 4.2 | 전자 잉크 표시<br>디스플레이 | DHT 11<br>온습도 센서 | 온도 습도를 측정 | L9110S 5V<br>모터       | 선풍기용 모터          |
| -                       | -                 | HS-CDSM<br>조도센서  | 빛의 세기를 측정 | 팬틸트<br>브라켓            | 각도 조절            |
|                         |                   | PIR 센서           | 움직임을 감지   | 서보모터 2개               | 각도 조절            |
|                         |                   | -                | -         | 건전지<br>케이스            | AA 배터리 4개        |
|                         |                   |                  |           | HG-ESP32<br>Devkit v3 | 선풍기 보드           |
|                         |                   |                  |           | 저항                    | 1K옴 3개 , 220K 1개 |

### 6.2 기존 설계 내용과 다르게 C 보드를 추가한 이유

먼저 A 보드를 통해 소프트웨어적인 설계와 deep sleep의 동작 원리를 이해하였다.

특히 deep sleep 상태에서는 CPU와 WiFi가 정지되어  
외부로부터 WiFi 기반의 인터럽트(데이터 수신)를 받을 수 없다는 한계를 직접 확인하였다.  
즉 A 보드가 잠들어 있는 동안에는 외부에서 능동적으로 데이터를 밀어 넣을 수 없고,  
A가 스스로 깨어나 요청하는 방식(pull)만 가능하다는 점이다.

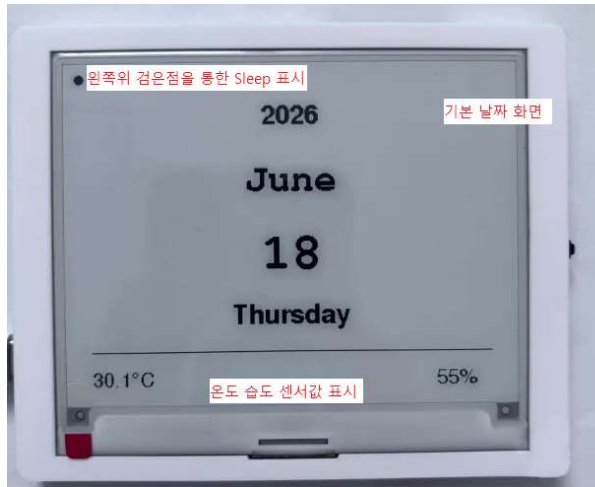
이를 통해 저전력 운용과 실시간 통신 사이의 트레이드오프,  
다시 말해 "깊게 잠들수록 전력은 아끼지만 외부 이벤트에 즉각 반응하기 어렵다"는 관계를 명확히 이해하게 되었다.

위의 내용과 같이 소프트웨어 설계적인 설계를 이해하였지만 하드웨어적인 이해가 부족하다고 느꼈다.  
A 보드의 소프트웨어 중심 학습을 넘어 모터·서보·초음파·LED 등 실제 액추에이터를 제어하는 하드웨어적 이해를 보완하기 위해 C보드를 추가했다.

또한 이를 통해 , B 보드가 단순히 A에게 데이터를 전달하는 중계기를 넘어, 자신이 보유한 여러  
센서(온도·조도·PIR)를 또 다른 노드인 C에게도 동시에 공급하는 진정한 의미의 허브로서 기능함을 보여 주기  
위함이다.

### 6.3 최종 구현 시스템 사진 등을 포함한 시스템 설명

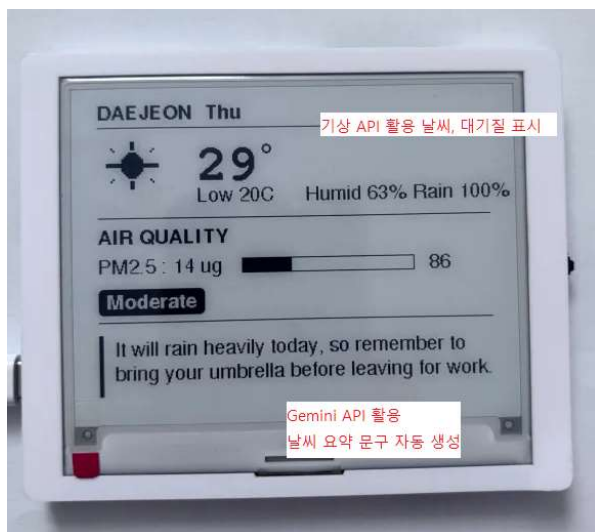
- A 보드



- 어떤화면에서든 10초 동작이 없으면 기본 날짜 화면으로 돌아오고 Deep Sleep
- 왼쪽 위 검은점을 통한 Sleep 유무를 확인
- B 보드에서 측정한 온도 습도 센서 표시



- 기본 날짜 화면의 월간 캘린더뷰

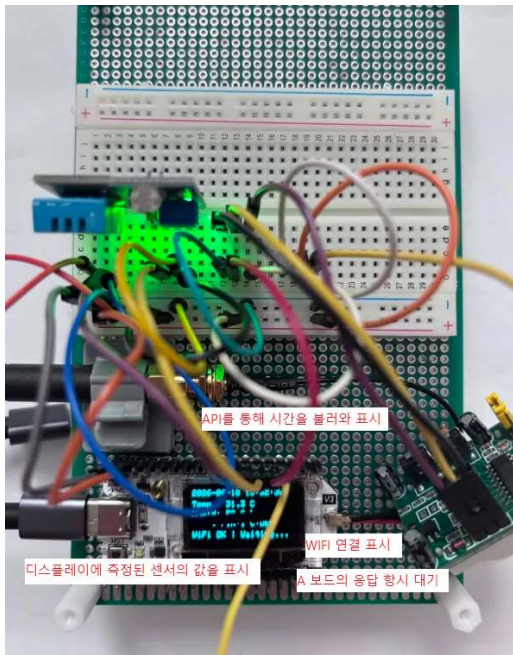


- 날씨 및 대기질
- 간단한 날씨 요약 문구
- 모든 정보는 A 보드에서 처리하지 않고 B보드에서 가져옴. A 보드는 화면 처리만

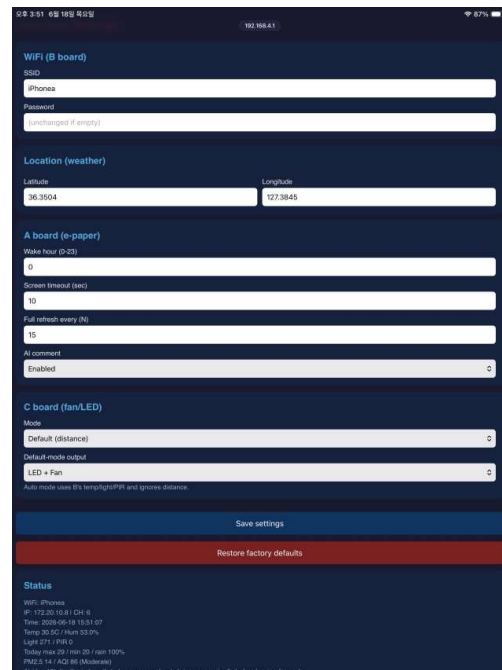


- 외부 API를 활용한 시간별 날씨

- B 보드

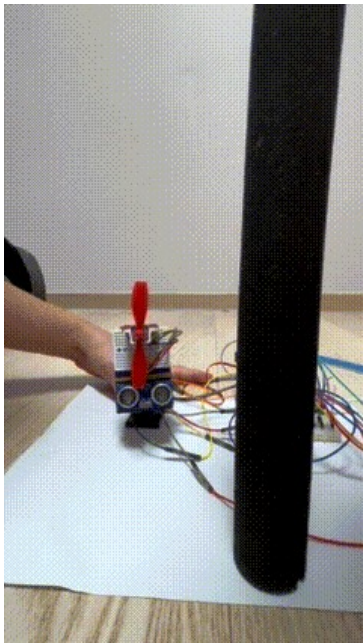


- 외부 API를 활용한 날씨 및 대기질 정보 수집
- 외부 API에서 가져온 수치를 Gemini에 보내 간단한 날씨 요약 문구 자동 생성
- 모든 정보를 만들어 A보드가 요청하면 정보를 보냄

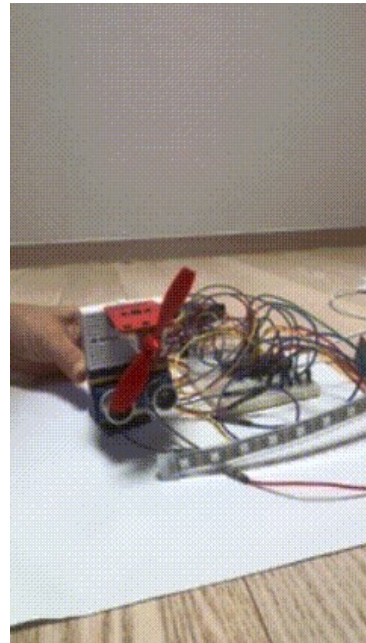


- B 보드 웹 접속하여 A 보드 및 C 보드 설정 변경

- C 보드 \* 해당 사진은 GIF 파일로 hwp에서 볼 수 있음 (확인이 불가할 경우 6.4의 링크)



- 스캔 후 가장 가까운 방향 바라도록 하였음
- 가장 가까운 거리를 저장하고 거리 안에 있으면 켜짐



- 가까이 가면 켜지고 멀어지면 꺼짐



## 6.4 동작 모습의 동영상

- A 보드의 자동 기상 및 날짜 변경 [https://youtube.com/shorts/PSg\\_g3vpkMU?feature=share](https://youtube.com/shorts/PSg_g3vpkMU?feature=share)
- C 보드 동작 (가까운 거리 인식) <https://youtube.com/shorts/GsqZDVRT7vI?feature=share>
- C 보드 동작 (거리 안 없을 경우 끄) [https://youtube.com/shorts/xn5pVVI\\_fzg?feature=share](https://youtube.com/shorts/xn5pVVI_fzg?feature=share)

다음 페이지에서 계속

## 7. 성능평가

### 7.1. (추정) 성능 실험

#### [주의]

아래의 내용은 성능 실험을 진행할 경우 적용할 수 있는 측정 방법에 대해 논하며  
성능 실험은 예상 전력 소모량 계산으로 대체하였다.

A 보드는 전자종이의 쌍안정성과 deep sleep을 활용한 저전력 단말이므로, 배터리 수명이 핵심 성능 지표이다. 실험 목적은 A 보드가 한 번 충전으로 며칠을 운용할 수 있는지를 전류 소비 측면에서 정량적으로 평가하는 것이다.

실험 구성은 다음과 같다. A 보드에 리튬 배터리(기준 용량 3000mAh 가정)를 연결하고, USB 전류계 또는 멀티미터를 전원 라인에 직렬로 연결해 세 가지 상태의 전류를 측정한다. 측정 상태는 첫째 deep sleep 대기 상태(화면 표시 유지, 통신·CPU 정지), 둘째 wakeup 후 ESP-NOW 통신 상태(B에 요청·수신), 셋째 전자종이 갱신 상태(display 갱신 중)이다.

실험 시나리오는 코드의 실제 동작을 기준으로 한다. A 보드는 평소 deep sleep으로 머물다가 하루에 정기 기상 1회(아침 wakeHour)와 사용자 수동 조작 약 5회, 합계 하루 약 6회 깨어나는 것을 표준 사용 패턴으로 가정한다. 한 번 깨어날 때 통신·갱신·대기시간(sleepTimeoutSec=10초)을 합쳐 약 15초간 활성 상태에 있다고 본다.

### 7.2. (추정) 실험 데이터 및 분석

| 동작 상태               | 전류 소비            | 지속 시간(1회) | 측정 방법       |
|---------------------|------------------|-----------|-------------|
| deep sleep 대기       | 약 20~100 $\mu$ A | 하루 대부분    | 전류계 직렬 측정   |
| wakeup + ESP-NOW 통신 | 약 80~120 mA      | 약 1~5초    | 통신 구간 평균 전류 |
| 전자종이 갱신(부분)         | 약 20~30 mA       | 약 0.3~1초  | 갱신 구간 측정    |
| 전자종이 갱신(전체)         | 약 20~30 mA       | 약 2~3초    | 15회마다 1회 발생 |
| 정지 화면 유지            | 약 0 $\mu$ A      | —         | 전원 무관       |

하루 소비 전력량을 추정하면 다음과 같다.

활성 상태(하루 6회  $\times$  약 15초  $\approx$  90초  $\approx$  0.025시간)에서 평균 100mA를 소비한다고 보면 활성 소비는 약 100mA  $\times$  0.025h  $\approx$  2.5mAh이다. deep sleep 상태(하루 약 23.97시간)에서 50 $\mu$ A(0.05mA)를 소비한다고 보면 대기 소비는 약 0.05mA  $\times$  23.97h  $\approx$  1.2mAh이다. 따라서 하루 총 소비는 약 3.7mAh로 추정된다.

이를 3000mAh 배터리에 적용하면 이론 수명은  $3000 \div 3.7 \approx$  약 810일이며, 자가방전·효율 손실·온도 등을 고려한 실질 안전계수를 적용해도 약 1년이상의 운용이 가능하다는 결론이 나온다.

7.3. 정량 및 정성 평가

- 정량 평가:

| 비교 항목        | 종이 캘린더                   | 본 시스템 (A 보드)        |
|--------------|--------------------------|---------------------|
| 현재 날짜 표시     | 불가능                      | 가능                  |
| 표시 가능 정보 종류  | 날짜·일정                    | 날짜·날씨·대기질·온습도·AI 멘트 |
| 실시간 정보 갱신    | 불가능                      | 가능                  |
| 사용자 수동 관리 횟수 | 월 1회 페이지 넘기기,<br>날짜 표시 X | 0회(자동 기상·갱신)        |
| 전력 소비(상시 표시) | 0                        | 약 3.7mAh/일          |
| 무충전 운용 기간    | 무한(종이)                   | 약 1년 이상             |
| 정보 해석·조언 제공  | 불가능                      | AI 한 줄 조언 제공        |
| 환경 연동(날씨·먼지) | 불가능                      | 자동 연동               |

- 정성 평가

| 항목                 | 시스템 A<br>(CrowPanel e-paper) | 시스템 B<br>(Heltec 게이트웨이)      | 시스템 C<br>(선풍기)        |
|--------------------|------------------------------|------------------------------|-----------------------|
| 경제성                | ○<br>(완제 패널, 무료 API)         | △<br>(상시전원·API 과금 가능)        | ○<br>(저가 부품)          |
| 목표 효과(정보<br>표시·제어) | ○<br>(달력·날씨·AI 멘트 표시)        | ○<br>(데이터 수집·중계 허브)          | ○<br>(팬·LED·서보 제어)    |
| 저전력성               | ○<br>(deep sleep+쌍안정 화면)     | ×<br>(WiFi 상시 구동)            | ×<br>(모터·서보 상시 구동)    |
| 통신 신뢰성             | ○<br>(채널스캔·재시도·캐시)           | ○<br>(NVS·버전동기화)             | ○<br>(페일세이프 OFF)      |
| 확장성                | ○<br>(다른 여러기능 추가 가능성)        | ○<br>(웹설정·다중노드 배포)           | ○<br>(다른 여러기능 추가 가능성) |
| 인터넷/클라우드<br>연동     | -<br>(직접연결 없음 -> 효율 증가)      | ○<br>(Open-Meteo-Gemini-NTP) | -<br>(직접연결 없음)        |
| 표시 가독성             | ○<br>(반사형·종이느낌)              | △<br>(소형 OLED)               | -<br>(관계 없음)          |
| 자동화/지능화            | △<br>(표시 위주 트레이드 오프)         | ○<br>(센서퓨전 판단)               | ○<br>(모드별 자동제어)       |
| 안정성(통신두절<br>대응)    | ○<br>(CACHED 폴백)             | ○ (재시도·캐시)                   | ○<br>(stale 시 안전정지)   |

## 8. 현실적 제한조건 반영

| 제한조건  | 반영 내용(결과물이 현실 제한조건에 어떤 영향을 미치는지 설명함, 관계없으면 관계없다고 지적, 아니면 구체적으로 설명함)                                                                                                          | 관련성 |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 경제성   | 날씨·대기질 데이터는 무료 API(Open-Meteo)를 활용하여 구축 비용을 낮추었다. Gemini의 무료 사용량 제한으로 하루에 한번 문구를 생성하고 저장하여 사용하는 식으로 설계하였다. 이로 인해 잘못된 문구가 나오거나 날씨 데이터의 변동으로 문구가 맞지 않는 현상이 있었다.                | △   |
| 환경성   | 전자종이의 특성과 deep sleep 설계로 A 보드가 1회 충전만으로 장기간 운용되어 배터리 교체·전력 소모를 크게 줄였다. 종이를 매월 교체·폐기할 필요가 없어 자원 절감에도 기여한다.                                                                    | O   |
| 사회성   | 날씨·미세먼지 정보와 AI 생활 조언을 항상 제공하여 사용자의 일상 정보 접근성과 건강 관리(마스크 착용 등)에 도움을 준다. 별도 앱 조작 없이 정보가 표시되어 디지털 취약 계층도 쉽게 이용할 수 있다.                                                           | O   |
| 윤리성   | 해당 내용은 프로젝트와 관계 x                                                                                                                                                            | //  |
| 미학    | 선풍기(팬)·서보·초음파 센서·배선이 외부로 드러나 외관이 다소 정돈되지 못하며, 각 부품의 형상과 배치가 제각각이라 깔끔한 인상을 주기 어렵다. 또한 부품을 일체로 감싸는 맞춤 케이스 제작이 3D 모델링·치수 측정·출력 등과 같은 과정은 난이도가 높아, 현재는 외관 마감보다 기능 구현에 우선순위를 두었다. | △   |
| 보건/안전 | 전자종이는 발광형이 아니어서 눈 피로와 블루라이트 부담이 적다. C 보드는 통신 두절 시 모든 출력을 끄는 페일세이프를 적용해 모터·팬의 오작동을 방지한다.                                                                                      | O   |
| 내구성   | A 보드는 deep sleep으로 부품 부하가 적어 수명이 길고, 통신 실패 시 캐시 폴백과 채널 자동 스캔으로 불안정한 환경에서도 안정적으로 동작한다.                                                                                        | O   |
| 산업표준  | ESP-NOW(Espressif 표준 프로토콜), I2C·SPI 통신, WMO 날씨 코드, US AQI 대기질 기준, REST/JSON API 등 널리 쓰이는 표준 규격을 준수하여 호환성과 확장성을 확보하였다.                                                        | O   |

## 9. 팀 역할 소개

---

- 팀역할 소개 및 팀장의 평가 포함

| 팀원  | 역할                          | 평가   | 인   |
|-----|-----------------------------|------|-----|
| 최준서 | 프로젝트 설계 및 구현 / 발표           | 25%  | 최준서 |
| 김가온 | 프로젝트 설계 및 구현 / 결과보고서 및 발표자료 | 25%  | 김가온 |
| 오성준 | 프로젝트 설계 및 구현 / 발표           | 25%  | 오성준 |
| 홍민혁 | 프로젝트 설계 및 구현 / 결과보고서 및 발표자료 | 25%  | 홍민혁 |
| 합계  |                             | 100% |     |

## 10. 참고문헌

---

<https://github.com/G6EJD/ESP32-e-Paper-Weather-Display>  
<https://github.com/lmarzen/esp32-weather-epd>  
<https://elecrow.com/blog/how-to-make-an-esp32-weather-e-paper.html>  
<https://www.hackster.io/lmarzen/esp32-e-paper-weather-display-a2f444>  
<https://www.espressif.com/en/solutions/low-power-solutions/esp-now>  
[https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp\\_now.html](https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp_now.html)  
<https://ieeexplore.ieee.org/document/9573246/>  
[https://www.researchgate.net/publication/373470899\\_Indoor\\_Performance\\_Evaluation\\_of\\_ESP-NOW](https://www.researchgate.net/publication/373470899_Indoor_Performance_Evaluation_of_ESP-NOW)  
<https://pmc.ncbi.nlm.nih.gov/articles/PMC8627223/>  
<https://patents.google.com/patent/US5961804A/en>  
<https://patents.google.com/patent/US8797634B2/en>  
<https://www.pervasivedisplays.com/how-e-paper-works/fast-update-refresh/>

## 11. 부록

---

- 사용 코드 및 hwp (3조 깃 저장소)

<https://github.com/noctis-03/microprocessorr.git>